FedMATRIX-LoRA: Client-Aware Matrix-Level Adaptation for Resource-Constrained Federated Fine-Tuning

Anonymous Authors

***Abstract*—Federated fine-tuning must allocate limited training resources across clients with heterogeneous data and device capabilities. For low-rank adaptation (LoRA), this requires deciding not only adapter capacity but also which target projections receive updates. Grouping projections into indivisible Transformer blocks restricts this placement decision. A controlled study using client-local importance scores shows larger short-term validation-loss reductions with independent matrix selection than with whole-layer allocation across six partition–checkpoint settings. Motivated by this observation, we present FedMATRIX-LoRA, a client-aware matrix-level adaptation framework. Clients compute matrix importance scores from local gradients and construct resource-feasible matrix configurations; the server coordinates participation and configuration choice under a shared budget and latency objective. Matrix-wise aggregation combines the resulting heterogeneous updates. A conditional stationarity analysis characterizes the roles of update bias and matrix coverage. Experiments spanning image classification, language understanding, and dialogue summarization evaluate the complete framework, while controlled studies examine selection granularity, scoring reliability, and the contributions of individual components. The results support importance-guided matrix selection as a useful adaptation dimension under constrained training budgets.**

***Index Terms*—Federated fine-tuning, low-rank adaptation, client-aware matrix selection, resource heterogeneity**

I. Introduction

Large pretrained models are increasingly adapted to tasks whose training data are distributed across organizations or devices. Federated learning (FL) enables these participants, referred to as clients, to improve a shared model without pooling their raw data [3], [4]. However, updating all model weights can exceed the memory, computation, and communication capacities of individual clients. Low-rank adaptation (LoRA) reduces this burden by freezing the pretrained backbone and training compact low-rank factors attached to selected weight projections [5]. Federated LoRA therefore reduces the trainable footprint, but leaves an important resource-allocation question: which adapter parameters should each client update?

A common way to accommodate heterogeneous clients is to adjust the LoRA rank, namely the inner dimension of its trainable low-rank factors. Lower ranks reduce adapter parameters and associated optimizer states. Nevertheless, training memory also includes intermediate activations required for backpropagation, whose cost does not necessarily decrease proportionally with rank. Fed-HeLLo addresses this distinction through heterogeneous allocation of trainable LoRA layers according to client resources and layer importance [8]. This shifts the allocation decision from how much capacity to assign within an adapter to which locations in the network should receive updates.

Layer-level allocation is a useful step toward selective adaptation, but its granularity can still constrain the choice of trainable parameters. A Transformer block contains distinct projections, including Query (Q), Key (K), and Value (V), each of which can receive an independent LoRA update. Previous studies have shown that adaptation value need not be uniform across projections or depths [5], [38]. When these projections are bound into an indivisible layer-level choice, updating one useful projection also consumes budget on the others in that group. Matrix-level selection relaxes this coupling by allowing useful projections to be selected across different blocks. This raises a natural question: under the same importance criterion and training budget, does independent matrix selection improve adaptation over whole-layer grouping?

To examine this question, we compare independent matrix selection with whole-layer allocation using the same client-local importance scores. Both schemes activate 12 rank-8 Q/K/V adapters on CIFAR100/ViT-base, but the former selects individual projections whereas the latter selects four complete Q/K/V groups. The comparison covers three data partitions and two starting checkpoints, with matched short-term training runs. As shown in Fig. 1, matrix-level selection achieves larger held-out validation-loss reductions in all six partition–checkpoint means and all 18 seed-level paired means. This observation motivates target-matrix granularity as a practical dimension of resource-constrained adaptation.

Independent selection introduces a further question: which matrices should a particular client train? Although clients share the current global adapter, their local data may induce different sensitivities to the same target projection. We therefore use a client-specific matrix importance score, computed from the squared gradients of the local loss, to estimate the relative value of candidate updates. This lightweight assessment uses data already available on each client and does not require an additional server-held proxy dataset. The scores guide which target projections deserve the limited training budget.

Importance scores alone, however, do not determine which updates can actually be executed. Clients have different memory capacities and training speeds, while the server must coordinate their decisions under a shared round budget and synchronization deadline. The resulting problem therefore couples two decisions: estimating matrix importance from each client's data and translating these scores into feasible training configurations

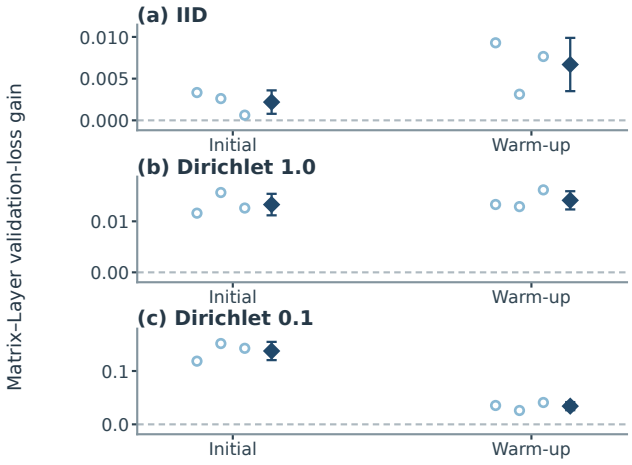


Fig. 1. Matrix-level versus whole-layer allocation using client-local importance scores. Positive values indicate larger held-out validation-loss reduction from matrix-level selection. Open circles denote individual seed means averaged over five clients; filled diamonds and error bars indicate the three-seed mean and sample standard deviation. Panel scales differ.

that can be coordinated across clients.

We propose FedMATRIX-LoRA to address these coupled decisions. Each client computes matrix importance scores using a local mini-batch, applies position-aware calibration, and generates candidate configurations satisfying its resource limits. The server jointly selects client configurations under the shared budget and latency objective. Selected clients train their assigned LoRA factors, and matrix-wise aggregation combines updates from potentially different client subsets into the global adapter. The framework thus connects fine-grained, client-aware target selection with resource-feasible federated training.

Our contributions are threefold:

- We formulate importance-guided target-matrix selection for resource-constrained federated fine-tuning, allowing projections within a Transformer block to be selected independently. A matched-budget study with client-local scoring demonstrates the benefit of independently selecting target projections over whole-layer allocation.
- We develop FedMATRIX-LoRA to connect client-local matrix importance scoring with resource-feasible configuration generation, shared-budget coordination, and matrix-wise aggregation. Clients can update distinct target subsets while contributing to a common global adapter.
- We provide a conditional stationarity analysis of selective aggregation and evaluate the framework across image classification, language understanding, and dialogue summarization. Controlled scoring and ablation studies examine the mechanisms separately from the complete system's performance.

Section II positions the work relative to prior studies. Section III defines the system model and allocation problem. Section IV presents the framework and its analysis, and Section V reports the evaluation. Section VI concludes the paper.

II. Related Work

Parameter-efficient and importance-guided adaptation. Parameter-efficient fine-tuning reduces the parameters updated when adapting a pretrained model. LoRA expresses updates through low-rank factors attached to selected weight matrices, while QLoRA combines a quantized frozen backbone with trainable adapters [5], [10]. These mechanisms address different components of the adaptation footprint. Selecting the target subset introduces a further decision about where the trainable capacity is placed, rather than only its size or storage precision.

Importance-aware adaptation has already established that a limited parameter budget need not be distributed uniformly. AdaLoRA assigns nonuniform rank budgets across weight matrices through importance-guided pruning of an SVD-style update parameterization [38]. FISH selects a sparse trainable-parameter mask using Fisher-based scores [39]. FedMATRIX-LoRA builds on importance-guided selection at a different allocation level: it chooses fixed-rank target subsets for individual federated clients. Whereas importance-aware rank allocation controls capacity within weight updates, the proposed formulation coordinates independently selectable target projections under heterogeneous client and round-level constraints.

Heterogeneous federated LoRA allocation. HetLoRA supports different client ranks through local rank self-pruning and sparsity-weighted aggregation, while FAH-QLoRA jointly considers quantization and LoRA rank [6], [7]. These approaches adapt capacity or numerical precision to client resources. Fed-HeLLo identifies an additional memory concern: reducing rank primarily affects optimizer-related state in the activation-dominated settings it studies, motivating partial-layer training [8]. LEGEND jointly configures tuning depth and rank distribution, explicitly linking trainable locations to resource use [32]. Target selection and rank allocation are consequently complementary dimensions, rather than mutually exclusive alternatives.

Fed-HeLLo accounts for unequal layer importance through server-proxy scores and geometric allocation strategies [8]. Its score-guided strategy converts a shared assessment into allocation probabilities, so clients can receive different layer masks. FedMATRIX-LoRA instead permits separate choices among projections within a block and evaluates the scoring criterion on client data. The controlled study in Fig. 1 compares matrix-level and whole-layer selection using identical client-local scores and matched parameter budgets. It isolates the allocation-granularity decision rather than reproducing the full Fed-HeLLo framework.

Fed-pilot is a closely related importance-guided allocation method. Its block-indexed formulation combines local and aggregated gradient-based information-gain scores with memory-constrained allocation and compensated aggregation [40]. FedMATRIX-LoRA exposes individual target-projection indicators within blocks and coordinates their configurations across clients under memory limits, latency constraints, and a shared round budget. The relevant distinction is the allocation decision and its coupled constraints, not local scoring or knapsack

optimization alone.

Aggregation and resource coordination. FedEx-LoRA addresses the mismatch associated with aggregating low-rank factors, and LoRA-FAIR refines aggregation and initialization [16], [17]. These studies concern how local updates are combined. Oort, FedBalancer, and PyramidFL address participant selection or training-work coordination under heterogeneous conditions [26]–[28]. FedMATRIX-LoRA includes the trainable target subset in the configuration assigned to a participating client. Configuration generation connects local update priorities to resource feasibility, while server coordination determines which configurations can execute together. The matrix-wise convergence analysis characterizes the selective update pattern produced by this process.

III. System Model and Problem Formulation

A. System Model and Federated LoRA Preliminaries

We consider a heterogeneous federated LoRA fine-tuning system consisting of one server and N clients. Each client i owns a private local dataset $\mathcal{D}_i$, an on-device training-memory capacity $M_i^{\max}$ (corresponding to the available GPU device memory in our implementation), an estimated communication bandwidth B_i, and heterogeneous computational resources. To preserve data privacy, all local datasets remain on their corresponding clients throughout the training process and are never uploaded to the server. The server initializes a pretrained backbone model Θ^0, which remains frozen during federated fine-tuning, and maintains the global LoRA adapter parameters θ^k at communication round k.

Let $n_i = |\mathcal{D}_i|$ denote the number of local training samples at client i. Federated LoRA fine-tuning aims to minimize the following global objective

$$F(\theta) = \frac{1}{N}\sum_{i=1}^{N} F_i(\theta), \tag{1}$$

where

$$F_i(\theta) = \frac{1}{n_i}\sum_{z\in\mathcal{D}_i} \ell(\Theta^0, \theta; z), \tag{2}$$

and $\ell(\cdot)$ denotes the task loss evaluated using the frozen backbone Θ^0 and trainable LoRA parameters θ.

For a target linear transformation $W_j \in \mathbb{R}^{d_{\text{out}}\times d_{\text{in}}}$, LoRA parameterizes its update as

$$\Delta W_j = B_j A_j, \tag{3}$$

where $A_j \in \mathbb{R}^{r\times d_{\text{in}}}$ and $B_j \in \mathbb{R}^{d_{\text{out}}\times r}$ are low-rank matrices with rank $r \ll \min(d_{\text{in}}, d_{\text{out}})$.

We treat the trainable adapter subset as a client-specific decision. A target matrix denotes a pretrained projection, such as Query, Key, or Value; selecting it means updating its associated LoRA factors, not the frozen pretrained matrix itself. The rank of every selected adapter is fixed to the same value, so configuration differences arise from target selection.

B. Client-Aware Matrix-Level LoRA Allocation

We formulate federated LoRA allocation at the target-matrix granularity. Let

$$\mathcal{A} = \{W_1, W_2, \ldots, W_L\} \tag{4}$$

denote the ordered set of LoRA-compatible target matrices extracted from the backbone. These matrices include the Query, Key, Value, output, and feed-forward projection matrices in Transformer blocks.

For client i at communication round k, the matrix activation decision is represented by the binary vector

$$\mathbf{m}_i^k = [m_{i,1}^k, \cdots, m_{i,L}^k], \tag{5}$$

where

$$m_{i,j}^k = \begin{cases} 1, \text{if } W_j \text{ is activated for local training;} \\ 0, \text{otherwise.} \end{cases} \tag{6}$$

Each activation vector therefore specifies the subset of LoRA target matrices trained by client i during communication round k.

Let $V_i^k(\mathbf{m}_i^k)$ denote the profiled utility of client i's matrix configuration. It is computed from local data at the current global adapter and summarizes the estimated priority of the selected updates. We set $V_i^k(\mathbf{0}) = 0$. The profiling procedure is defined in Section IV-B.

Layer-group allocation is recovered by constraining all candidate matrices in a Transformer block to share the same selection indicator.

C. Resource Constraints and Problem Formulation

The activation decision of LoRA target matrices determines the resource consumption of each participating client. We model the training cost, training-memory usage, and training latency separately.

First, let $c_i(\mathbf{m}_i^k)$ denote the normalized training cost associated with matrix configuration $\mathbf{m}_i^k$. We approximate this cost by the number of activated rank-8 matrices, $c_i(\mathbf{m}_i^k) = \sum_{j=1}^{L} m_{i,j}^k$. This standardized count is not a measurement of matrix-specific computation or memory, which can differ even at the same rank; memory is modeled separately below.

Training-memory consumption depends on both the number and positions of activated matrices. For any nonzero $\mathbf{m}$, let $\mathcal{J}(\mathbf{m}) = \{j \mid m_j = 1\}$ and $j_{\min}(\mathbf{m}) = \min \mathcal{J}(\mathbf{m})$. Following the training-memory decomposition commonly adopted in partial-layer federated LoRA fine-tuning [8], we set $M_i(\mathbf{0}) = M_i^{\text{fixed}}$, while a nonzero configuration uses

$$M_i(\mathbf{m}) = M_i^{\text{fixed}} + q_i(r)c_i(\mathbf{m}) + \sum_{j=j_{\min}(\mathbf{m})}^{L} \mu_{i,j}, \tag{7}$$

where M_i^{fixed} denotes the memory occupied by the frozen backbone and fixed runtime buffers, $q_i(r)$ is the parameter-and-optimizer memory per activated rank-r LoRA matrix, and $\mu_{i,j}$ is retained activation memory. The last term reflects the position-dependent memory characteristic of Transformer

fine-tuning: activating an earlier matrix requires retaining a longer backward propagation path and therefore consumes more activation memory.

The latency of client i consists of local computation and communication. Let γ_i denote its computation coefficient, σ_i the communication size per activated matrix, and B_i its available bandwidth. Its latency is

$$\begin{aligned} T_i^{\mathrm{comp}}(\mathbf{m}_i) &= \gamma_i c_i(\mathbf{m}_i), \\ T_i^{\mathrm{comm}}(\mathbf{m}_i) &= \frac{\sigma_i c_i(\mathbf{m}_i)}{B_i}, \\ T_i(\mathbf{m}_i) &= T_i^{\mathrm{comp}}(\mathbf{m}_i) + T_i^{\mathrm{comm}}(\mathbf{m}_i). \end{aligned} \tag{8}$$

The coefficients define an analytical resource profile; equal matrix-equivalent costs need not correspond to identical measured hardware costs.

At communication round k, the server jointly determines the participating clients and their matrix configurations under a shared resource budget. Let S_k denote the set of candidate clients, $z_i^k \in \{0,1\}$ indicate whether client i participates in the current round, C^k denote the round-level training budget, and τ^k represent the latency of the current communication round. Let $\ell_i^{\min}$ and $\ell_i^{\max}$ bound the number of matrices activated by participating client i, and let $\lambda > 0$ control the utility–latency trade-off.

$$\max_{\{\mathbf{m}_i^k, z_i^k\}, \tau^k} \quad \sum_{i \in S_k} V_i^k(\mathbf{m}_i^k) - \lambda \tau^k \tag{9}$$

$$\text{s.t.} \quad \sum_{i \in S_k} c_i(\mathbf{m}_i^k) \le C^k, \tag{10}$$

$$m_{i,j}^k \le z_i^k, \tag{11}$$

$$\ell_i^{\min} z_i^k \le \sum_{j=1}^{L} m_{i,j}^k \le \ell_i^{\max} z_i^k, \tag{12}$$

$$M_i(\mathbf{m}_i^k) \le M_i^{\max}, \quad \forall i : z_i^k = 1, \tag{13}$$

$$T_i(\mathbf{m}_i^k) \le \tau^k \le T_{\max}, \quad \forall i : z_i^k = 1. \tag{14}$$

The objective maximizes the aggregated client-conditioned adaptation utility while minimizing round latency. Equations (10)–(14) impose the round-level matrix-equivalent budget, participation consistency, activated-matrix bounds, per-client training-memory limits, and the round deadline. In particular, Eqs. (11) and (12) together ensure that an inactive client selects no matrix, whereas an active client selects between $\ell_i^{\min}$ and $\ell_i^{\max}$ matrices.

The formulation couples client-level feasibility with a shared round budget. We implement it by constructing feasible configurations locally and coordinating these configurations at the server.

IV. FedMATRIX-LoRA Design

A. Framework Overview

FedMATRIX-LoRA implements importance-guided matrix selection through client-conditioned assessment and resource-feasible allocation. Figure 2 separates these roles. Clients compute matrix importance scores using local data and form configurations of independently selectable targets. The server coordinates the configurations under the round budget and latency objective. Clients then update the assigned factors, and their contributions are aggregated matrix-wise into the next shared adapter. The following subsections specify the scoring rule, allocation procedure, training protocol, and conditional analysis.

B. Client-Conditioned Matrix Importance Scoring

Local matrix importance. Matrix-level allocation requires an importance score for each candidate projection. At the current global adapter, gradients of a client's local loss describe the sensitivity of that client's objective to the associated LoRA parameters. We use these gradients to score candidate matrices before selecting a trainable subset. The raw score and its depth-calibrated counterpart represent two forms of the same importance estimate; configuration utility subsequently combines the scores of selected matrices.

A single-pass gradient score. Gradient- and Fisher-based criteria have been used to select trainable parameters and LoRA layers [8], [39]. We use a gradient-squared criterion that evaluates all candidate targets in one backward pass, rather than trial-training individual configurations. During a refresh at round k, client i attaches LoRA modules to the candidate projections, loads the current global adapter θ^k, and evaluates the average loss on a local mini-batch $\mathcal{B}_i^k$. Let θ_j^k denote the LoRA factors associated with target matrix j. Its raw score is

$$g_{i,j}^k = \sum_{p \in \theta_j^k} \left(\nabla_p \mathcal{L}_i(\theta^k; \mathcal{B}_i^k) \right)^2. \tag{15}$$

The score aggregates the magnitude of the batch-loss gradient over the adapter's trainable coordinates. Squaring makes coordinate contributions nonnegative; summation yields a block-level sensitivity measure rather than a mean per parameter. For a fixed parameterization, the gradient norm measures the largest first-order sensitivity to a unit-norm perturbation of that block. This motivates using its square as an update-priority signal. The calculation uses the gradient of a batch-averaged loss, so the score is a Fisher-inspired proxy rather than exact Fisher information. Its empirical relation to short-term training gains is evaluated in Section V-D.

Both the local examples and the current adapter enter the score. Clients can therefore assign different priorities to a common target, and refreshes allow those priorities to evolve with the global model. Each profiling pass evaluates the candidate set, including targets not updated by that client in the preceding round. The next allocation is consequently informed by more than the client's previously active subset. The single-pass design supplies this information without a separate optimization run for every candidate configuration.

Depth-calibrated importance scores. Raw gradient scores capture local sensitivity but do not explicitly account for the position of a target projection within the network. Motivated

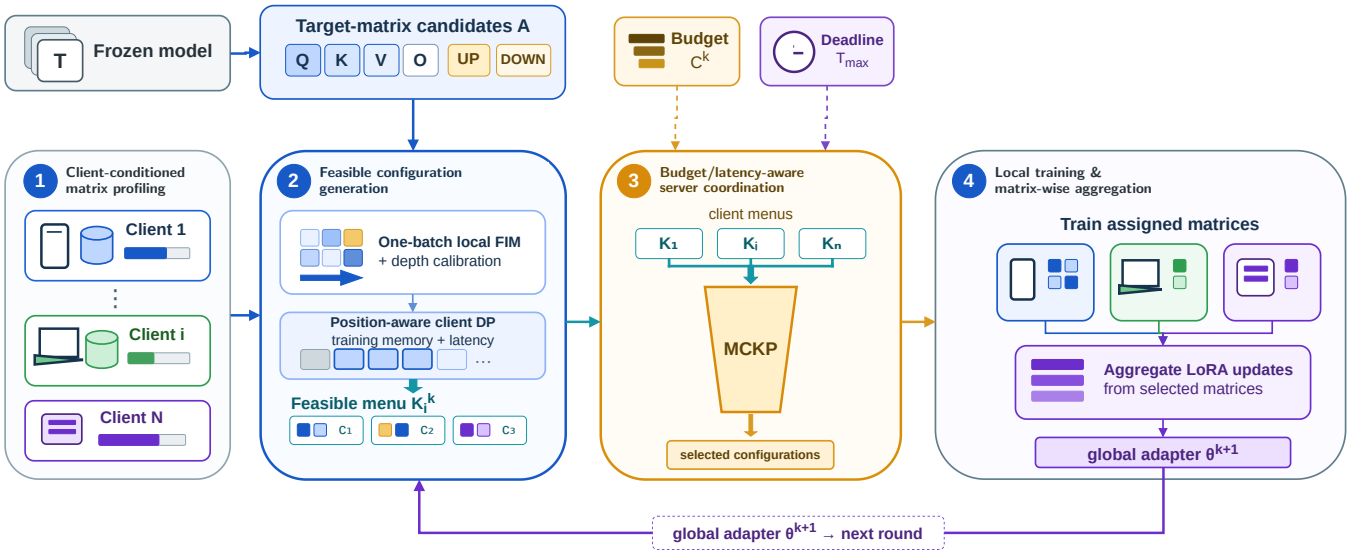


Fig. 2. FedMATRIX-LoRA workflow: client-conditioned profiling, local feasible configuration generation, budget- and latency-aware server coordination, and local training with matrix-wise aggregation.

by prior evidence on depth-dependent adaptation and resource trade-offs [32], we introduce a multiplicative positional factor:

$$\widetilde{g}^k_{i,j} = g^k_{i,j}(1 + d_j)^\rho, \tag{16}$$

where $d_j = (j-1)/(L-1)$ denotes the normalized target position and ρ controls the strength of the positional preference. The multiplier adjusts each client's gradient-based importance scores without imposing a common selection mask. Its contribution is examined separately in Section V-E.

Configuration utility and refresh. Server coordination compares configurations proposed by different clients. The calibrated scores therefore use a common normalization scale:

$$G^k_{\max} = \max_{\substack{h\in S_k \\ 1\le j\le L}} \widetilde{g}^k_{h,j}, \qquad v^k_{i,j} = \frac{\widetilde{g}^k_{i,j}}{\max\{G^k_{\max}, \delta\}} \in [0,1], \tag{17}$$

where $\delta > 0$ prevents division by zero. A common divisor retains relative calibrated magnitudes across clients and matrices and sets the utility scale in the server's utility–latency objective. The assignment utility is $V^k_i(\mathbf{m}^k_i) = \sum_{j=1}^{L} m^k_{i,j} v^k_{i,j}$. This additive score supplies a tractable selection objective; it does not require the full training loss to decompose into independent matrix losses. Local evidence, the positional preference, and the common scale have separate roles in constructing that objective.

Profiles are cached between designated refresh rounds and recomputed when missing or due for refresh. Reuse avoids an additional backward pass at every allocation decision; refreshing reconnects the scores to the current adapter. Raw examples and profiling gradients remain local, while coordination uses matrix scores and configuration summaries. Section V-D evaluates the ranking information supplied by the score, and Section V-E distinguishes scoring-source, granularity, and calibration effects within the complete training pipeline.

C. Resource-Feasible Matrix Allocation

For a fixed round k, superscripts are omitted from configuration attributes and scheduling variables when unambiguous. Each feasible configuration κ generated by client i is represented as

$$\kappa = (V_{i\kappa}, c_{i\kappa}, T_{i\kappa}, M_{i\kappa}, \mathbf{m}_{i\kappa}),$$

where $V_{i\kappa}$, $c_{i\kappa}$, $T_{i\kappa}$, $M_{i\kappa}$, and $\mathbf{m}_{i\kappa}$ denote the profiled adaptation utility, normalized matrix-equivalent cost, estimated latency, estimated training-memory consumption, and binary target-matrix assignment, respectively. Each client uploads a compact configuration menu $\mathcal{K}^k_i$.

Local configurations. Under the adopted memory model, feasibility depends on the selected-matrix count and the earliest selected position through the retained-activation term. For each count c, the client therefore searches over feasible earliest positions and retains a configuration with the largest profiled utility. Let $Q^k_{i,e}(t)$ denote the maximum utility of selecting t matrices from the suffix $\{e,\dots,L\}$. Its recursion and the utility for configurations whose earliest selected position is e are: $Q^k_{i,e}(0) = 0$ and $Q^k_{i,L+1}(t>0) = -\infty$.

$$\begin{aligned} Q^k_{i,e}(t) &= \max\left\{Q^k_{i,e+1}(t), v^k_{i,e} + Q^k_{i,e+1}(t-1)\right\}, \quad t \ge 1, \\ \Phi^k_{i,e}(c) &= v^k_{i,e} + Q^k_{i,e+1}(c-1). \end{aligned} \tag{18}$$

Only configurations satisfying the matrix-count, memory, and latency constraints are retained. Under the current count-based latency model, configurations with the same cost consume the same server budget and have the same latency; retaining the highest-utility feasible configuration at each cost therefore suffices for subsequent coordination.

Server coordination. The server enumerates the distinct feasible latency thresholds $\tau \le T_{\max}$. For each threshold, it removes configurations exceeding τ and solves the remaining multiple-choice knapsack problem, choosing at most one

configuration per client. With $H_i(w)$ denoting the largest utility attainable from the first i client menus at budget w, the recursion is:

$$H_i(w) = \max_{\kappa \in \mathcal{K}_i^k(\tau) \cup \{\varnothing\} : c_{i\kappa} \le w} \{H_{i-1}(w - c_{i\kappa}) + V_{i\kappa}\}, \quad (19)$$

Here $\mathcal{K}_i^k(\tau)$ is the menu after latency filtering, $\varnothing$ is the zero-utility inactive choice with zero cost, and $H_0(w) = 0$.

The server selects the threshold and configuration assignment with the largest total utility minus $\lambda\tau$. This procedure optimizes the stated profiled objective; its role is to realize client-conditioned selection under the resource model. Algorithm 1 summarizes the complete training round.

D. Local Training and Matrix-Wise Aggregation

After determining the optimal configuration assignment, the server dispatches the global adapter θ^k together with the selected matrix assignments to corresponding clients. Only selected LoRA parameters are updated locally. Task-specific trainable modules, if required by the downstream task, are optimized jointly with the selected LoRA parameters.

Since different clients may update different subsets of LoRA matrices, the aggregation process is performed independently for each trainable matrix. Let $\mathcal{U}_j^k$ denote the clients returning updates for matrix j, and let $\theta_{i,j}^{k+1}$ be the corresponding locally updated LoRA parameters from client i. The server performs matrix-wise client-uniform aggregation:

$$\theta_j^{k+1} = \begin{cases} |\mathcal{U}_j^k|^{-1} \sum_{i \in \mathcal{U}_j^k} \theta_{i,j}^{k+1}, & |\mathcal{U}_j^k| > 0, \\ \theta_j^k, & |\mathcal{U}_j^k| = 0. \end{cases} \quad (20)$$

The updated global adapter is used to initialize the next round. Because the updating client set can differ across target matrices, the aggregation averages only the clients contributing to each matrix and preserves the previous value when no update is available.

E. Convergence of Matrix-Wise Aggregation

We analyze the effective update induced by the matrix-wise aggregation rule. For this analysis, θ includes the LoRA parameters and any jointly trained shared task modules. Let P^k project onto the parameter blocks selected for update in round k, as determined before local training. For a common positive round scale γ, define the actual aggregated direction by $v^k = (\theta^k - \theta^{k+1})/\gamma$. Conditioning on the current model, profiling information, and selected configuration, denoted by $\mathcal{H}_k$, write

$$b^k = \mathbb{E}[v^k \mid \mathcal{H}_k] - P^k \nabla F(\theta^k), \quad (21)$$

$$\xi^k = v^k - \mathbb{E}[v^k \mid \mathcal{H}_k]. \quad (22)$$

Thus $\mathbb{E}[\xi^k \mid \mathcal{H}_k] = 0$, and the update is

$$\theta^{k+1} = \theta^k - \gamma\big(P^k \nabla F(\theta^k) + b^k + \xi^k\big). \quad (23)$$

The bias b^k accounts for selective aggregation and the difference between local optimizer updates and the selected global gradient. This representation does not replace the experimental AdamW optimizer with SGD.

Algorithm 1 FedMATRIX-LoRA in communication round k

Require: Global adapter θ^k, candidate matrix set $\mathcal{A}$, candidate clients S_k, budget constraint C^k, latency bound $T_{\max}$

Client-conditioned matrix profiling and allocation:
for all $i \in S_k$ **do**
  if profiling cache is unavailable or refresh is required **then**
    Insert LoRA modules on candidate matrices $\mathcal{A}$
    Compute matrix importance scores $g_{i,j}^k$ using one local mini-batch
    Apply position-aware calibration to obtain $\widetilde{g}_{i,j}^k$
  else
    Reuse cached matrix importance scores
  end if
  Normalize matrix utilities to obtain $v_{i,j}^k$
  Generate feasible configuration menu $\mathcal{K}_i^k$ via suffix dynamic programming
end for

Resource-feasible server coordination:
Initialize best objective value $U^* \leftarrow -\infty$
for all distinct feasible latency thresholds $\tau \le T_{\max}$ **do**
  Remove configurations satisfying $T_{i\kappa} > \tau$
  Solve MCKP using dynamic programming under budget constraint C^k
  Compute objective value $\sum_i \sum_\kappa x_{i\kappa} V_{i\kappa} - \lambda\tau$
  if current objective $> U^*$ **then**
    Update the best configuration assignment
  end if
end for

Local training and matrix-wise aggregation:
Dispatch selected matrix assignments and θ^k to clients
for all selected clients **do**
  Train LoRA parameters only on assigned matrices
end for
Perform matrix-wise aggregation to obtain θ^{k+1}
return updated global adapter θ^{k+1}

Assumptions. Assume that F is β-smooth on the relevant update segments and bounded below by $F_{\inf}$. Suppose the effective update errors satisfy

$$\mathbb{E}\|b^k\|^2 \le B_{\text{upd}}^2, \qquad \mathbb{E}\|\xi^k\|^2 \le S_{\text{upd}}^2. \quad (24)$$

We additionally assume effective matrix coverage: for some $0 < \alpha_{\text{cov}} \le 1$ and $\varepsilon_{\text{cov},k} \ge 0$,

$$\mathbb{E}\|P^k \nabla F(\theta^k)\|^2 \ge \alpha_{\text{cov}} \mathbb{E}\|\nabla F(\theta^k)\|^2 - \varepsilon_{\text{cov},k}. \quad (25)$$

This condition quantifies the global gradient energy captured by the selected blocks.

Theorem 1 (Expected stationarity)**.** *Under the stated assumptions, for $0 < \gamma \le 1/\beta$ and K rounds,*

$$\frac{1}{K} \sum_{k=0}^{K-1} \mathbb{E}\|\nabla F(\theta^k)\|^2 \le \frac{2D_0}{\alpha_{\text{cov}} \gamma K} + \frac{B_{\text{upd}}^2 + \beta\gamma S_{\text{upd}}^2 + \bar{\varepsilon}_{\text{cov}}}{\alpha_{\text{cov}}}, \quad (26)$$

where $D_0 = F(\theta^0) - F_{\inf}$ for a fixed initialization and $\bar{\varepsilon}_{\text{cov}} = K^{-1} \sum_k \varepsilon_{\text{cov},k}$.

For a fixed horizon, setting $\gamma = c/\sqrt{K} \le 1/\beta$ yields an $O(K^{-1/2})$ optimization and noise term when the error bounds

TABLE I
EVALUATION SETTINGS. NON-IID x: DIRICHLET $\alpha = x$; LEN-M/S: MILD/SEVERE LENGTH SKEW; SLASH-SEPARATED BUDGETS FOLLOW PARTITION ORDER.

Dataset	Task	Backbone	Partitions	Rounds	Cum. budget	Metric
CIFAR100	Image classification	ViT-base [2]	IID, Non-IID 1.0/0.1	180	7,200/12,000	Accuracy
MNLI	Language inference	DeBERTa-v3-large [36]	IID, Non-IID 1.0	50	9,500	Accuracy
QNLI	Sentence-pair classification	RoBERTa-base [37]	IID, Non-IID 1.0	80	4,800/7,200	Accuracy
QQP	Paraphrase classification	DeBERTa-v3-large [36]	IID, Non-IID 1.0	50	9,500	Accuracy
SST-2	Sentiment classification	RoBERTa-base [37]	IID, Non-IID 1.0	80	7,200	Accuracy
SAMSum	Dialogue summarization	OPT-1.3B [1]	IID, Len-M, Len-S	40	10,000	Rouge-L

and coverage constant are uniform across the analyzed runs. The residual is governed by effective-update bias and uncovered gradient energy. The result characterizes average stationarity, rather than optimal test accuracy.

The contribution of client selection can be made explicit. Let $n_j = |\mathcal{U}_j| > 0$, $g_{i,j} = \nabla_j F_i(\theta)$, $g_j = N^{-1} \sum_i g_{i,j}$, and $\zeta_j^2 = N^{-1} \sum_i \|g_{i,j} - g_j\|^2$.

Lemma 1 (Matrix-wise selection bias)**.** *For any updating-client set $\mathcal{U}_j$,*

$$\left\| \frac{1}{n_j} \sum_{i \in \mathcal{U}_j} g_{i,j} - g_j \right\|^2 \leq \left(\frac{N}{n_j} - 1 \right) \zeta_j^2. \tag{27}$$

The bound relates aggregation bias to participation and gradient heterogeneity at each target matrix. Proofs are provided in Appendix A.

V. EXPERIMENTS

We evaluate the complete framework on the main benchmark settings and use controlled studies to distinguish selection granularity, scoring reliability, and component effects. Resource-efficiency comparisons use the stated analytical client profiles.

A. Evaluation Setup

We evaluate image classification on CIFAR100 [33], language understanding on MNLI, QNLI, QQP, and SST-2 from GLUE [34], and dialogue summarization on SAMSum [35].

Table I gives the backbones, partitions, rounds, and rank-8 matrix-cost budgets; each round has ten clients. Classification uses equal random IID partitions or Dirichlet non-IID partitions with $\alpha = 1.0$ or 0.1. SAMSum uses contiguous length-sorted shards for severe skew and randomly assigned length-sorted shards for mild skew.

Federated runs use NVIDIA A800-SXM4-80GB GPUs. Table II gives the analytical client resource profiles and training settings. The main comparisons use seed 2024 with shared partitions and resource draws. The additional scoring and ablation studies report their seed sets and data splits separately. Cumulative training cost $\overline{C}^K = \sum_{k=1}^K \sum_i c_i(\mathbf{m}_i^k)$ excludes profiling. Elapsed-time curves accumulate the analytical latency in Eq. (8), not end-to-end wall-clock time. For positional calibration, we use $\rho = 1 + 1.5\mathbb{I}_{\text{pair}} + \eta$, where $\mathbb{I}_{\text{pair}}$ indicates sentence-pair tasks and $\eta = 0, 1, 2$ for IID, moderate non-IID, and severe non-IID settings, respectively.

TABLE II
SYSTEM AND TRAINING PROFILE.

Control	Reported configuration
Clients	10
Training-memory budget	40/25/15 GB; ratio 6:3:1
Compute factor	0.8–2.0 s per rank-8 matrix-equivalent
Bandwidth	5–50 MB/s
LoRA rank / profiling	8 / one local mini-batch
Local training cap	one epoch, at most 50 mini-batch steps
Optimizer	AdamW
Batch size / learning rate	1–32 / 10^{-5}–10^{-4}
Default latency cap	200 s per round

We report final and best Accuracy for classification and ROUGE-L for SAMSum, without averaging the two metrics.

We compare FedMATRIX-LoRA with representative baselines, including homogeneous LoRA training (FedAvg-LoRA) [3], [5], uniform matrix allocation (Uniform), layer-wise allocation (Fed-HeLLo) [8], layer-group heuristic allocation (MS), heterogeneous-rank allocation (LEGEND [32] and HETLoRA-style [6]), and joint quantization/rank allocation (FAH-QLoRA-style) [7]. These baselines represent different adaptation granularities and resource-allocation strategies, enabling a comprehensive comparison with our client-aware matrix scheduling. For a fair comparison, all methods use the same backbone models, data partitions, communication rounds, local training settings, and evaluation protocol.

B. Task Performance and Resource Efficiency

Fig. 3 compares the best observed task metric across 14 evaluation settings. As shown in Fig. 3, FedMATRIX-LoRA consistently ranks first across all CIFAR100, GLUE, and SAMSum settings, covering both IID and heterogeneous data partitions. Compared with heterogeneous-rank and quantization-based baselines, FedMATRIX-LoRA also delivers substantially higher peak performance. For example, on MNLI IID, HETLoRA-style and FAH-QLoRA-style achieve 0.796 and 0.808, respectively, whereas FedMATRIX-LoRA reaches 0.880. On CIFAR100 IID, the corresponding accuracies are 0.806, 0.810, and 0.900, showing the performance of the complete framework under the reported evaluation protocol.

The budget–quality trajectories in Fig. 4 characterize the performance obtained as the cumulative matrix-equivalent cost increases. Together with the matched-target comparison in

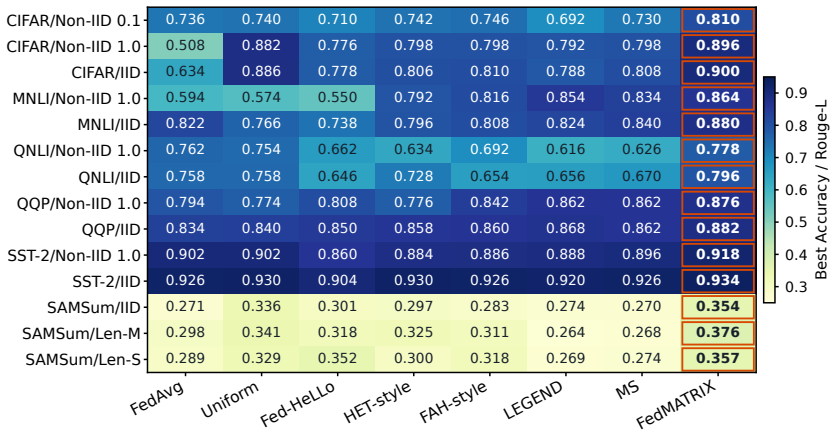


Fig. 3. Best task metric across 14 settings. Boxes mark row maxima; SAMSum uses Rouge-L and all other rows Accuracy.

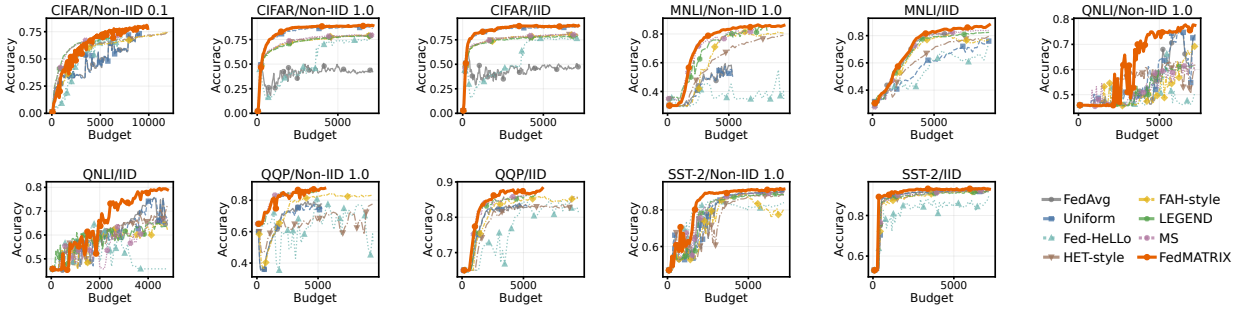


Fig. 4. Budget–quality trajectories for 11 classification scenarios; FedMATRIX-LoRA is emphasized.

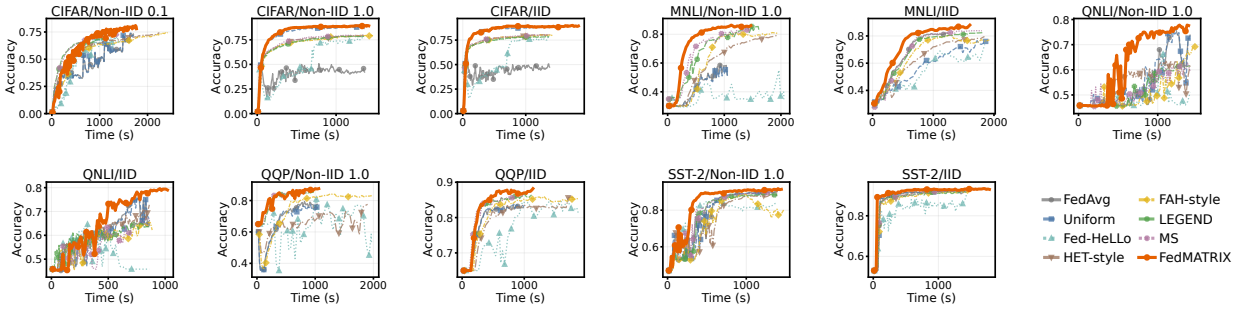


Fig. 5. Modeled-time–quality trajectories for all 11 classification scenarios under the same analytical client profiles as Fig. 4.

Table III, they quantify the resource trade-off of the complete framework.

Figure 5 reports the corresponding trajectories against cumulative modeled latency under the same analytical client profiles. The matched-target results in Table III summarize the modeled-time savings; these values are distinct from end-to-end measured runtime.

We additionally compare the resources required to reach the peak classification accuracy. Table III reports the cumulative-cost and modeled-time savings ($\Delta\overline{C}$ and ΔT) of FedMATRIX-LoRA relative to the least-resource baseline reaching the same target. Across all 11 classification scenarios, FedMATRIX-LoRA uses fewer resources, with median savings of 41.3% in cumulative budget and 42.3% in modeled elapsed time, demonstrating consistently higher resource efficiency across diverse workloads.

C. Controlled Study of Selection Granularity

We evaluate the effect of selection granularity on CIFAR100/ViT-base using 36 Q/K/V candidates across 12 Transformer blocks. Both schemes activate 12 rank-8 matrices: matrix-level allocation selects projections independently, whereas layer-level allocation selects four complete Q/K/V groups. Both use the same client-local scores, resource lim-

TABLE III
RESOURCES REQUIRED TO MATCH EACH STRONGEST BASELINE'S PEAK ACCURACY; POSITIVE SAVINGS FAVOR FEDMATRIX-LORA.

Scenario	Ref.	Target	$\Delta\overline{C}$ (%)	ΔT (%)
CIFAR / Non-IID 0.1	FAH-style	.746	+53.9	+59.4
CIFAR / Non-IID 1.0	Uniform	.882	+42.9	+42.4
CIFAR / IID	Uniform	.886	+54.4	+48.3
MNLI / Non-IID 1.0	LEGEND	.854	+5.6	+27.7
MNLI / IID	MS	.840	+33.2	+42.3
QNLI / Non-IID 1.0	FedAvg	.762	+0.7	+3.2
QNLI / IID	FedAvg	.758	+17.4	+3.2
QQP / Non-IID 1.0	LEGEND	.862	+14.9	+21.7
QQP / IID	LEGEND	.868	+41.3	+44.2
SST-2 / Non-IID 1.0	FedAvg	.902	+44.4	+45.2
SST-2 / IID	HET-style	.930	+47.8	+38.5

its, and training protocol. The comparison covers IID and Dirichlet-$1.0/0.1$ partitions, initialization and a shared warm-up checkpoint, and three random seeds. Each paired configuration undergoes 20 local training steps from the same starting adapter, with improvement measured on held-out client validation data.

We report the difference between the validation-loss reductions achieved by matrix-level and layer-level allocation, averaged over five clients within each seed. As shown in Fig. 1, matrix-level selection achieves positive mean gains in all six settings and positive seed-level paired gains in all 18 comparisons. This evidence supports treating target projections as independently selectable adaptation units under a fixed budget.

D. Scoring Reliability

We evaluate whether matrix importance scores can predict which training configurations will yield larger adaptation gains. For each client and checkpoint, we construct 12 configurations with the same parameter budget. Before training, each configuration is ranked using either the sum of its raw gradient-squared matrix scores or the sum of its depth-calibrated scores. We then train each configuration for 20 steps from a matched model state and rank the configurations by their actual reduction in held-out validation loss.

We measure the agreement between predicted and observed rankings using Spearman's rank correlation coefficient r_s. A value of 1 indicates identical rankings, whereas a value near 0 indicates little rank association. We compute r_s separately for each client's 12 configurations, average the coefficients across clients within each seed, and report the mean and sample standard deviation across three seeds.

Table IV shows that both scoring variants are positively correlated with actual adaptation gains across all six settings. Depth-calibrated scores achieve mean correlations of 0.657–0.876 and improve upon raw scores in five of the six settings. Raw scores perform better in the Dirichlet-0.1 initialization setting. These results support using gradient-based matrix importance to guide configuration selection, while positional calibration provides an additional, setting-dependent adjustment.

TABLE IV
SPEARMAN RANK CORRELATION (r_s) BETWEEN SCORE-PREDICTED CONFIGURATION RANKINGS AND OBSERVED 20-STEP VALIDATION-LOSS-REDUCTION RANKINGS. RAW USES GRADIENT-SQUARED SCORES; CALIBRATED ADDS POSITIONAL WEIGHTING. ENTRIES ARE THREE-SEED MEAN $\pm$ SAMPLE STANDARD DEVIATION AFTER CLIENT AVERAGING.

Partition	State	Raw (r_s)	Calibrated (r_s)
IID	Initial	0.688 ± 0.114	0.746 ± 0.067
IID	Warm-up	0.595 ± 0.094	0.657 ± 0.070
Dir. 1.0	Initial	0.792 ± 0.012	0.841 ± 0.042
Dir. 1.0	Warm-up	0.556 ± 0.141	0.664 ± 0.136
Dir. 0.1	Initial	0.885 ± 0.028	0.772 ± 0.028
Dir. 0.1	Warm-up	0.801 ± 0.023	0.876 ± 0.036

TABLE V
SST-2 COMPONENT ABLATION AT A CUMULATIVE MATRIX-EQUIVALENT BUDGET OF 7,200. VALIDATION ACCURACY IS REPORTED IN PERCENT AS MEAN $\pm$ SAMPLE STANDARD DEVIATION OVER THREE SEEDS. BEST DENOTES THE PEAK WITHIN EACH RUN. BOLD MARKS THE LARGEST MEAN IN EACH METRIC COLUMN.

Variant	Final Accuracy	Best Accuracy
Full	$\mathbf{90.60} \pm 0.72$	90.87 ± 0.50
Server-Proxy	90.47 ± 0.81	90.80 ± 0.87
Layer-Level	90.07 ± 1.33	90.73 ± 0.42
No-Depth	90.47 ± 0.61	$\mathbf{90.93} \pm 0.42$

E. Ablation of Matrix-Level Adaptation

We isolate three components using SST-2/RoBERTa-base with a Dirichlet-1.0 partition. Full is compared with Server-Proxy, which replaces local scores by a shared score computed on a reserved training-only proxy set; Layer-Level, which retains local scoring but ties Q/K/V choices within each block; and No-Depth, which removes only the positional multiplier. All variants use the same private-data and proxy splits within each seed, rank eight, local optimizer settings, candidate-client protocol, resource limits, and 80-round horizon. Their actual selected clients and target subsets remain outcomes of allocation. The three seeds are 2024–2026, and each run consumes a cumulative matrix-equivalent budget of 7,200. This controlled study has its own proxy reservation and is reported separately from the historical main comparison.

Table V summarizes the three-seed ablation results at an equal cumulative budget of 7,200. Full achieves the highest mean final accuracy (90.60%), exceeding Server-Proxy, Layer-Level, and No-Depth by 0.13, 0.53, and 0.13 percentage points, respectively. The largest reduction in mean final accuracy occurs when independent matrix selection is replaced by whole-layer allocation, consistent with the granularity study in Fig. 1. No-Depth achieves the highest mean best accuracy (90.93%). These comparisons distinguish the contributions of selection granularity, scoring source, and positional calibration.

VI. Conclusion

FedMATRIX-LoRA treats individual LoRA target projections as selectable adaptation units under heterogeneous client resources. A controlled study with client-local importance scores demonstrates larger short-term adaptation gains from independent matrix selection than from whole-layer binding across the evaluated partition and checkpoint settings. The framework evaluates priorities on local data, constructs feasible configurations, and coordinates their execution under a shared budget before matrix-wise aggregation. A conditional analysis relates the resulting selective updates to effective-update bias and matrix coverage. The main benchmarks and controlled ablations assess complementary aspects of this design: the complete framework's performance and resource trade-offs, and the effects of the scoring and allocation mechanisms. Together, these results motivate importance-guided target selection as a practical dimension of resource-constrained federated adaptation.

Appendix A
Proofs for Matrix-Wise Aggregation

Proof of Theorem 1. Condition on $\mathcal{H}_k$ and set $a = P^k \nabla F(\theta^k)$ and $\bar{v} = \mathbb{E}[v^k \mid \mathcal{H}_k] = a + b^k$. Because the update is supported on P^k, smoothness gives

$$\mathbb{E}[F(\theta^{k+1}) \mid \mathcal{H}_k] \le F(\theta^k) - \gamma \langle a, \bar{v} \rangle + \frac{\beta\gamma^2}{2} \left(\|\bar{v}\|^2 + \mathbb{E}[\|\xi^k\|^2 \mid \mathcal{H}_k] \right). \tag{28}$$

Use $2\langle a, \bar{v} \rangle = \|a\|^2 + \|\bar{v}\|^2 - \|b^k\|^2$ and $\gamma \le 1/\beta$ to discard the nonpositive term $-\gamma(1-\beta\gamma)\|\bar{v}\|^2/2$. Taking total expectations yields

$$\mathbb{E}F(\theta^{k+1}) \le \mathbb{E}F(\theta^k) - \frac{\gamma}{2}\mathbb{E}\|P^k \nabla F(\theta^k)\|^2 + \frac{\gamma}{2} B_{\mathrm{upd}}^2 + \frac{\beta\gamma^2}{2} S_{\mathrm{upd}}^2. \tag{29}$$

Summing over k telescopes the loss terms. The lower bound on F gives

$$\frac{1}{K} \sum_k \mathbb{E}\|P^k \nabla F(\theta^k)\|^2 \le \frac{2D_0}{\gamma K} + B_{\mathrm{upd}}^2 + \beta\gamma S_{\mathrm{upd}}^2. \tag{30}$$

Apply the coverage condition and divide by α_{cov}. □

Proof of Lemma 1. Let $d_i = g_{i,j} - g_j$ and $w_i = \mathbf{1}\{i \in \mathcal{U}_j\}/n_j$. Since $\sum_i d_i = 0$, the selection bias equals $\sum_i (w_i - 1/N) d_i$. Cauchy–Schwarz gives

$$\left\| \sum_i (w_i - 1/N) d_i \right\|^2 \le \sum_i (w_i - 1/N)^2 \sum_i \|d_i\|^2 \tag{31}$$

$$= (1/n_j - 1/N) N \zeta_j^2, \tag{32}$$

which is the claimed bound. For the full effective bias, define the remaining local-update discrepancy as $r = b - s$, with zero selection-bias blocks when no client updates a block. Then $\|b\|^2 \le 2\|s\|^2 + 2\|r\|^2$. The latter term includes local drift, optimizer effects, and any local-model mismatch; the selection lemma alone does not bound it. □

TABLE VI
Matrix-level minus layer-level validation-loss reduction using client-local scores. Positive values favor matrix-level selection. Entries are mean ± sample standard deviation across three seed-level client averages.

Partition	Checkpoint	Local gain
IID	Initial	$+0.00219 \pm 0.00140$
IID	Warm-up	$+0.00669 \pm 0.00319$
Dir. 1.0	Initial	$+0.01328 \pm 0.00211$
Dir. 1.0	Warm-up	$+0.01411 \pm 0.00179$
Dir. 0.1	Initial	$+0.13752 \pm 0.01708$
Dir. 0.1	Warm-up	$+0.03412 \pm 0.00757$

Appendix B
Complete Local-Score Granularity Results

Table VI provides the complete numerical results underlying Fig. 1. Each entry reports the additional held-out validation-loss reduction from matrix-level selection over whole-layer allocation using identical client-local scores. The differences are averaged across five clients within each seed and summarized as mean ± sample standard deviation over three seeds. All six setting-level mean differences are positive.

References

[1] S. Zhang *et al.*, "OPT: Open pre-trained transformer language models," *arXiv preprint arXiv:2205.01068*, 2022.

[2] A. Dosovitskiy *et al.*, "An image is worth 16x16 words: Transformers for image recognition at scale," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.

[3] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2017, pp. 1273–1282.

[4] P. Kairouz *et al.*, "Advances and open problems in federated learning," *Foundations and Trends in Machine Learning*, vol. 14, nos. 1–2, pp. 1–210, 2021, doi: 10.1561/2200000083.

[5] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2022.

[6] Y. J. Cho, L. Liu, Z. Xu, A. Fahrezi, and G. Joshi, "Heterogeneous LoRA for federated fine-tuning of on-device foundation models," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 2024, pp. 12903–12913, doi: 10.18653/v1/2024.emnlp-main.717.

[7] Z. Gao, Z. Zhang, Y. Guo, and Y. Gong, "Federated adaptive fine-tuning of large language models with heterogeneous quantization and LoRA," in *Proceedings of IEEE Conference on Computer Communications (INFOCOM)*, 2025, pp. 1–10, doi: 10.1109/INFOCOM55648.2025.11044641.

[8] Z. Zhang, P. Liu, J. Xu, and R. Hu, "Fed-HeLLo: Efficient federated foundation model fine-tuning with heterogeneous LoRA allocation," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 36, no. 10, pp. 17556–17569, 2025, doi: 10.1109/TNNLS.2025.3580495.

[9] Q. Li, B. He, and D. Song, "Model-contrastive federated learning," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021, pp. 10713–10722, doi: 10.1109/CVPR46437.2021.01057.

[10] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Advances in Neural Information Processing Systems*, vol. 36, 2023, pp. 10088–10115.

[11] N. Houlsby, A. Giurgiu, S. Jastrzebski, B. Morrone, Q. de Laroussilhe, A. Gesmundo, M. Attariyan, and S. Gelly, "Parameter-efficient transfer learning for NLP," in *Proceedings of the 36th International Conference on Machine Learning*, vol. 97, 2019, pp. 2790–2799.

[12] X. L. Li and P. Liang, "Prefix-tuning: Optimizing continuous prompts for generation," in *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, 2021, pp. 4582–4597, doi: 10.18653/v1/2021.acl-long.353.

[13] Z. Zhang, Y. Yang, Y. Dai, Q. Wang, Y. Yu, L. Qu, and Z. Xu, "FedPETuning: When federated learning meets the parameter-efficient tuning methods of pre-trained language models," in *Findings of the Association for Computational Linguistics: ACL 2023*, 2023, pp. 9963–9977, doi: 10.18653/v1/2023.findings-acl.632.

[14] J. Chen, W. Xu, S. Guo, J. Wang, J. Zhang, and H. Wang, "FedTune: A deep dive into efficient federated fine-tuning with pre-trained transformers," *arXiv preprint arXiv:2211.08025*, 2022.

[15] T. Che, J. Liu, Y. Zhou, J. Ren, J. Zhou, V. Sheng, H. Dai, and D. Dou, "Federated learning of large language models with parameter-efficient prompt tuning and adaptive optimization," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023, pp. 7871–7888, doi: 10.18653/v1/2023.emnlp-main.488.

[16] R. Singhal, K. Ponkshe, and P. Vepakomma, "FedEx-LoRA: Exact aggregation for federated and efficient fine-tuning of large language models," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2025, pp. 1316–1336, doi: 10.18653/v1/2025.acl-long.67.

[17] J. Bian, L. Wang, L. Zhang, and J. Xu, "LoRA-FAIR: Federated LoRA fine-tuning with aggregation and initialization refinement," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2025, pp. 3737–3746.

[18] Z. Li, B. Xu, X. Shu, J. Zhang, Y. Yao, G.-S. Xie, and J. Tang, "Tensor-aggregated LoRA in federated fine-tuning," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2025, pp. 1058–1067.

[19] H. Wang, X. Wang, Y. Li, J. Chen, J. Zhang, N. Yan, K. He, and W. Wang, "Federated LoRA fine-tuning with pipelined error-mitigated aggregation and matrix-wise freezing," in *Findings of the Association for Computational Linguistics: ACL 2026*, 2026, pp. 5749–5762, doi: 10.18653/v1/2026.findings-acl.284.

[20] J. Bai, D. Chen, B. Qian, L. Yao, and Y. Li, "Federated fine-tuning of large language models under heterogeneous tasks and client resources," in *Advances in Neural Information Processing Systems*, vol. 37, 2024, pp. 14457–14483, doi: 10.52202/079017-0461.

[21] J. Koo, M. Jang, and J. Ok, "Towards robust and efficient federated low-rank adaptation with heterogeneous clients," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2025, pp. 416–429, doi: 10.18653/v1/2025.acl-long.19.

[22] X. Huang, Y. Hu, Y.-J. Gong, and X. Zhang, "GMFL: Efficient global masking for federated LLM fine-tuning," in *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2026, pp. 25293–25312, doi: 10.18653/v1/2026.acl-long.1160.

[23] H. Ban and K. Ji, "Rethinking parameter sharing for LLM fine-tuning with multiple LoRAs," in *Findings of the Association for Computational Linguistics: ACL 2026*, 2026, pp. 12843–12863, doi: 10.18653/v1/2026.findings-acl.625.

[24] E. Diao, J. Ding, and V. Tarokh, "HeteroFL: Computation and communication efficient federated learning for heterogeneous clients," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.

[25] S. Alam, L. Liu, M. Yan, and M. Zhang, "FedRolex: Model-heterogeneous federated learning with rolling sub-model extraction," in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 29677–29690.

[26] F. Lai, X. Zhu, H. V. Madhyastha, and M. Chowdhury, "Oort: Efficient federated learning via guided participant selection," in *Proceedings of the 15th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2021, pp. 19–35.

[27] J. Shin, Y. Li, Y. Liu, and S.-J. Lee, "FedBalancer: Data and pace control for efficient federated learning on heterogeneous clients," in *Proceedings of the 20th Annual International Conference on Mobile Systems, Applications and Services (MobiSys)*, 2022, pp. 436–449, doi: 10.1145/3498361.3538917.

[28] C. Li, X. Zeng, M. Zhang, and Z. Cao, "PyramidFL: A fine-grained client selection framework for efficient federated learning," in *Proceedings of the 28th Annual International Conference on Mobile Computing and Networking (MobiCom)*, 2022, pp. 158–171, doi: 10.1145/3495243.3517017.

[29] Y. G. Kim and C.-J. Wu, "AutoFL: Enabling heterogeneity-aware energy efficient federated learning," in *Proceedings of the 54th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2021, pp. 183–198, doi: 10.1145/3466752.3480129.

[30] B. Luo, X. Li, S. Wang, J. Huang, and L. Tassiulas, "Cost-effective federated learning in mobile edge networks," *IEEE Journal on Selected Areas in Communications*, vol. 39, no. 12, pp. 3606–3621, 2021, doi: 10.1109/JSAC.2021.3118436.

[31] J. Kirkpatrick *et al.*, "Overcoming catastrophic forgetting in neural networks," *Proceedings of the National Academy of Sciences of the United States of America*, vol. 114, no. 13, pp. 3521–3526, 2017, doi: 10.1073/pnas.1611835114.

[32] J. Liu, Y. Liao, H. Xu, Y. Xu, J. Liu, and C. Qian, "Adaptive parameter-efficient federated fine-tuning on heterogeneous devices," *IEEE Transactions on Mobile Computing*, vol. 24, no. 11, pp. 12533–12549, 2025, doi: 10.1109/TMC.2025.3586644.

[33] A. Krizhevsky, "Learning multiple layers of features from tiny images," Department of Computer Science, University of Toronto, Toronto, ON, Canada, Technical Report, 2009.

[34] A. Wang, A. Singh, J. Michael, F. Hill, O. Levy, and S. R. Bowman, "GLUE: A multi-task benchmark and analysis platform for natural language understanding," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2019.

[35] B. Gliwa, I. Mochol, M. Biesek, and A. Wawer, "SAMSum corpus: A human-annotated dialogue dataset for abstractive summarization," in *Proceedings of the 2nd Workshop on New Frontiers in Summarization*, 2019, pp. 70–79, doi: 10.18653/v1/D19-5409.

[36] P. He, X. Liu, J. Gao, and W. Chen, "DeBERTa: Decoding-enhanced BERT with disentangled attention," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.

[37] Y. Liu *et al.*, "RoBERTa: A robustly optimized BERT pretraining approach," *arXiv preprint arXiv:1907.11692*, 2019.

[38] Q. Zhang, M. Chen, A. Bukharin, N. Karampatziakis, P. He, Y. Cheng, W. Chen, and T. Zhao, "AdaLoRA: Adaptive budget allocation for parameter-efficient fine-tuning," *arXiv preprint arXiv:2303.10512*, version 2, 2023.

[39] Y.-L. Sung, V. Nair, and C. Raffel, "Training neural networks with fixed sparse masks," in *Advances in Neural Information Processing Systems*, vol. 34, 2021.

[40] Z. Zhang, R. Hu, P. Liu, and J. Xu, "Fed-pilot: Optimizing LoRA allocation for efficient federated fine-tuning with heterogeneous clients," *arXiv preprint arXiv:2410.10200*, version 2, 2025.